

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Darshan N (1BM24CS089)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Darshan N (**1BM24CS089**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Mayanka Gupta
Assoistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	7
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	10
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	12
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	15
6	Implement 0/1 Knapsack problem using dynamic programming.	18
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	21
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	23
9	Implement Fractional Knapsack using Greedy technique.	28
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	30
11	Implement "N-Queens Problem" using Backtracking.	32

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

#define M 20

int main() {
    int n, adj[M][M];
    int indegree[M] = {0};
    int visited[M] = {0};
    int i, j, count = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(adj[i][j] == 1) {
                indegree[j]++;
            }
        }
    }

    printf("Topological Order: ");

    while(count < n) {
        int found = 0;

        for(i = 0; i < n; i++) {
            if(indegree[i] == 0 && visited[i] == 0) {

                printf("%d ", i);

                visited[i] = 1;
            }
        }
    }
}
```

```

        found = 1;
        count++;

    /* Reduce indegree of adjacent vertices */
    for(j = 0; j < n; j++) {
        if(adj[i][j] == 1) {
            indegree[j]--;
        }
    }
}

if(found == 0) {
    printf("\nTopological ordering not possible (Graph contains cycle)\n");
    return 0;
}

return 0;
}

```

Output: leetcode 207

Problem List < > 207

Submissions

Status	Language	Runtime	Memory	Notes
Accepted	C++	23 ms	109.9 MB	

Accepted
Mar 09, 2025

Suggested complexity: $O(V + E)$
Suggestion: Replace the adjacency matrix with an adjacency list to reduce complexity from quadratic to linear.

```

Code | C
1 bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int
2 int** graph = (int**)malloc(numCourses * sizeof(int*));
3 int* indegree = (int*)calloc(numCourses, sizeof(int));
4 int* graphColSize = (int*)calloc(numCourses, sizeof(int));
5 for (int i = 0; i < numCourses; i++) {
6     graph[i] = (int*)malloc(numCourses * sizeof(int));
7 }
8
9 for (int i = 0; i < prerequisitesSize; i++) {
10     int course = prerequisites[i][0];
11     int prereq = prerequisites[i][1];
12     graph[prereq][graphColSize[prereq]++] = course;
13     indegree[course]++;
14 }
15
16 int* queue = (int*)malloc(numCourses * sizeof(int));
17 int front = 0, rear = 0;
18
19 // Add courses with in-degree 0
20 for (int i = 0; i < numCourses; i++) {
21     if (indegree[i] == 0) {
22         queue[rear++] = i;
23     }
24 }

```

Testcase | Test Result

Output

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

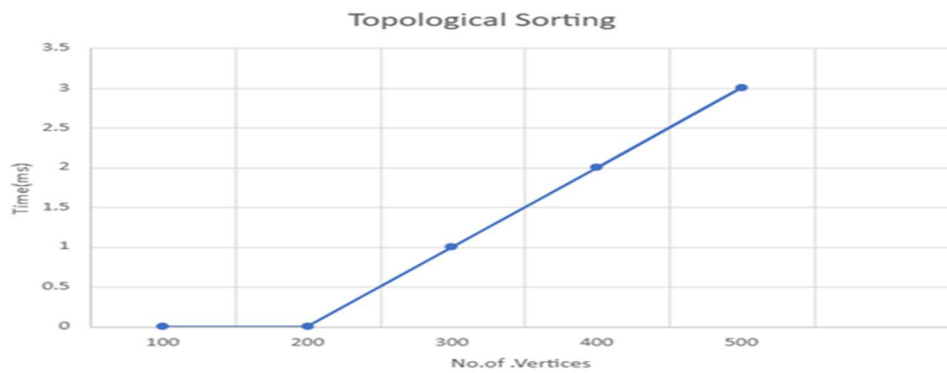
0 0 0 1

0 0 0 1

0 0 0 0

Topological Order: 0 1 2 3

GRAPH:



LAB PROGRAM 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
```

```
#define MAX 20
```

```
void prt(int a[], int n) {  
    int i;  
    for(i = 0; i < n; i++)  
        printf("%d ", a[i]);  
    printf("\n");  
}
```

```
int mobile(int a[], int dir[], int n) {  
    int i;  
    int mobile_prev = 0;  
    int mobile_index = -1;  
  
    for(i = 0; i < n; i++) {  
  
        if(dir[i] == 0 && i != 0) {  
            if(a[i] > a[i - 1] && a[i] > mobile_prev) {  
                mobile_prev = a[i];  
                mobile_index = i;  
            }  
        }  
  
        if(dir[i] == 1 && i != n - 1) {  
            if(a[i] > a[i + 1] && a[i] > mobile_prev) {  
                mobile_prev = a[i];  
                mobile_index = i;  
            }  
        }  
    }  
  
    return mobile_index;  
}
```

```
int main() {  
    int n, i, j;  
    int a[MAX], dir[MAX];
```

```
printf("Enter number of elements: ");  
scanf("%d", &n);
```

```
for(i = 0; i < n; i++) {  
    a[i] = i + 1;  
    dir[i] = 0;  
}
```

```
printf("Permutations:\n");  
prt(a, n);
```

```
while(1) {  
  
    int m = mobile(a, dir, n);  
  
    if(m == -1)  
        break;  
  
    int swap_pos;  
  
    if(dir[m] == 0)  
        swap_pos = m - 1;  
    else  
        swap_pos = m + 1;  
  
    int temp = a[m];  
    a[m] = a[swap_pos];  
    a[swap_pos] = temp;  
  
    temp = dir[m];  
    dir[m] = dir[swap_pos];  
    dir[swap_pos] = temp;  
  
    m = swap_pos;  
  
    for(i = 0; i < n; i++) {  
        if(a[i] > a[m]) {  
            dir[i] = 1 - dir[i];  
        }  
    }  
  
    prt(a, n);  
}
```

```
    return 0;  
}
```

OUTPUT:

Output

Enter number of elements: 3

Permutations:

1 2 3

1 3 2

3 1 2

3 2 1

2 3 1

2 1 3

LAB PROGRAM 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define MAX 100
```

```
void merge(int a[], int low, int mid, int high) {
```

```
    int i, j, k;
```

```
    int temp[MAX];
```

```
    i = low;
```

```
    j = mid + 1;
```

```
    k = low;
```

```
    while(i <= mid && j <= high) {
```

```
        if(a[i] < a[j])
```

```
            temp[k++] = a[i++];
```

```
        else
```

```
            temp[k++] = a[j++];
```

```
    }
```

```
    while(i <= mid)
```

```
        temp[k++] = a[i++];
```

```
    while(j <= high)
```

```
        temp[k++] = a[j++];
```

```
    for(i = low; i <= high; i++)
```

```
        a[i] = temp[i];
```

```
}
```

```
void ms(int a[], int low, int high) {
```

```
    int mid;
```

```
    if(low < high) {
```

```
        mid = (low + high) / 2;
```

```
        ms(a, low, mid);
```

```
        ms(a, mid + 1, high);
```

```
        merge(a, low, mid, high);
```

```
}
```

```

}

int main() {
    int a[MAX], n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    start = clock();

    ms(a, 0, n - 1);

    end = clock();

    time_taken = ((double)(end - start))

    printf("Sorted elements are:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    printf("\nTime taken = %f seconds\n", time_taken);

    return 0;
}

```

OUTPUT:

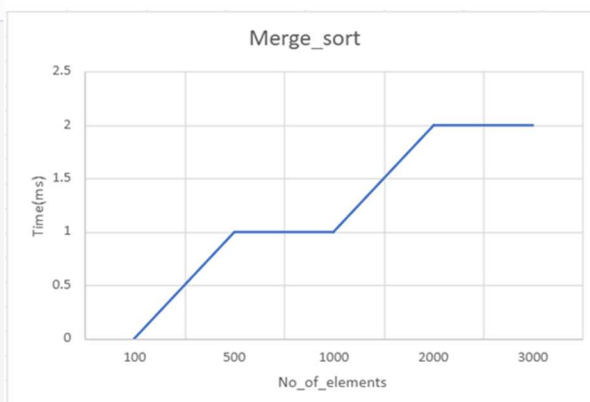
Output

```

Enter number of elements: 7
Enter elements:
11 15 13 1 2 4 7
Sorted elements are:
1 2 4 7 11 13 15
Time taken = 3.000000 seconds

```

=== Code Execution Successful ===



LAB PROGRAM 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
#include <time.h>

#define MAX 100

int part(int a[], int low, int high) {
    int pivot, i, j, temp;

    pivot = a[low];
    i = low + 1;
    j = high;

    while(i <= j) {

        while(i <= high && a[i] <= pivot)
            i++;

        while(a[j] > pivot)
            j--;

        if(i < j) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    temp = a[low];
    a[low] = a[j];
    a[j] = temp;

    return j;
}

void quick(int a[], int low, int high) {
    int p;

    if(low < high) {
        p = part(a, low, high);
```

```

        quick(a, low, p - 1);
        quick(a, p + 1, high);
    }
}

int main() {
    int a[MAX], n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    start = clock();

    quick(a, 0, n - 1);

    end = clock();

    time_taken = ((double)(end - start))
    printf("Sorted elements are:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    printf("\nTime taken = %f seconds\n", time_taken);

    return 0;
}

```

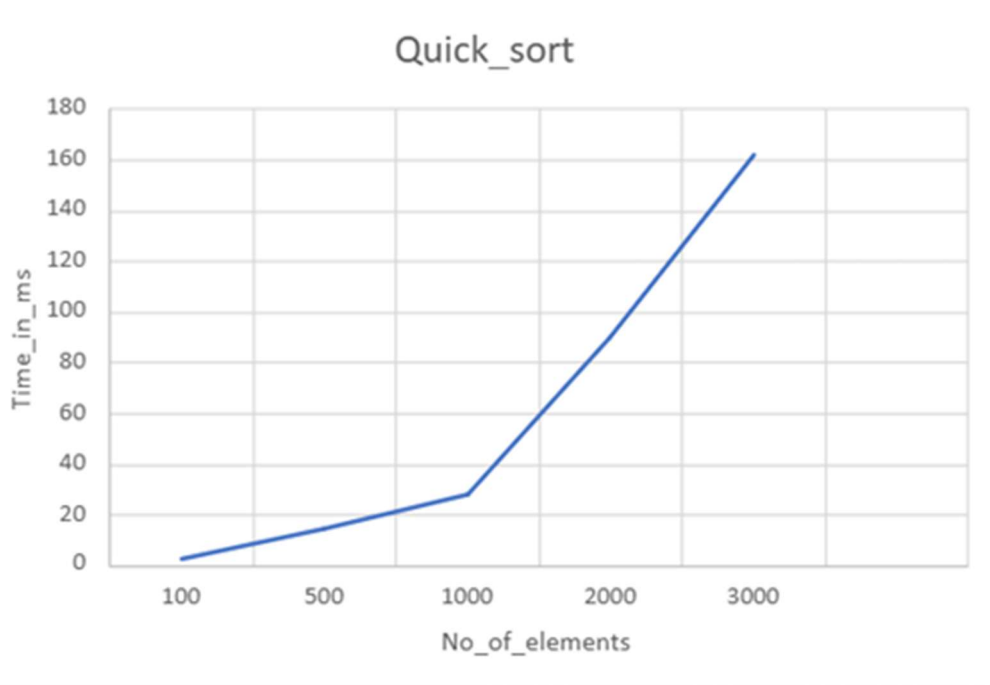
OUTPUT:

Output

```
Enter number of elements: 7
Enter elements:

11
15
13
1
2
4
7
Sorted elements are:
1 2 4 7 11 13 15
Time taken = 0.000002 seconds
```

GRAPH :



LAB PROGRAM 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define MAX 100
```

```
void heapify(int a[], int n, int i) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
    int temp;  
  
    if(left < n && a[left] > a[largest])  
        largest = left;  
  
    if(right < n && a[right] > a[largest])  
        largest = right;  
  
    if(largest != i) {  
        temp = a[i];  
        a[i] = a[largest];  
        a[largest] = temp;  
  
        heapify(a, n, largest);  
    }  
}
```

```
void buildheap(int a[], int n) {  
    int i;  
  
    for(i = n / 2 - 1; i >= 0; i--) {  
        heapify(a, n, i);  
    }  
}
```

```
void heapsort(int a[], int n) {  
    int i, temp;  
  
    buildheap(a, n);  
  
    for(i = n - 1; i > 0; i--) {
```

```

        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

void printarr(int a[], int n) {
    int i;

    for(i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    printf("\n");
}

int main() {
    int a[MAX], n, i;
    clock_t start, end;
    double time_taken;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    start = clock();

    heapsort(a, n);

    end = clock();

    time_taken = ((double)(end - start)) ;

    printf("Sorted elements are:\n");
    printarr(a, n);

    printf("Time taken = %f seconds\n", time_taken);
}

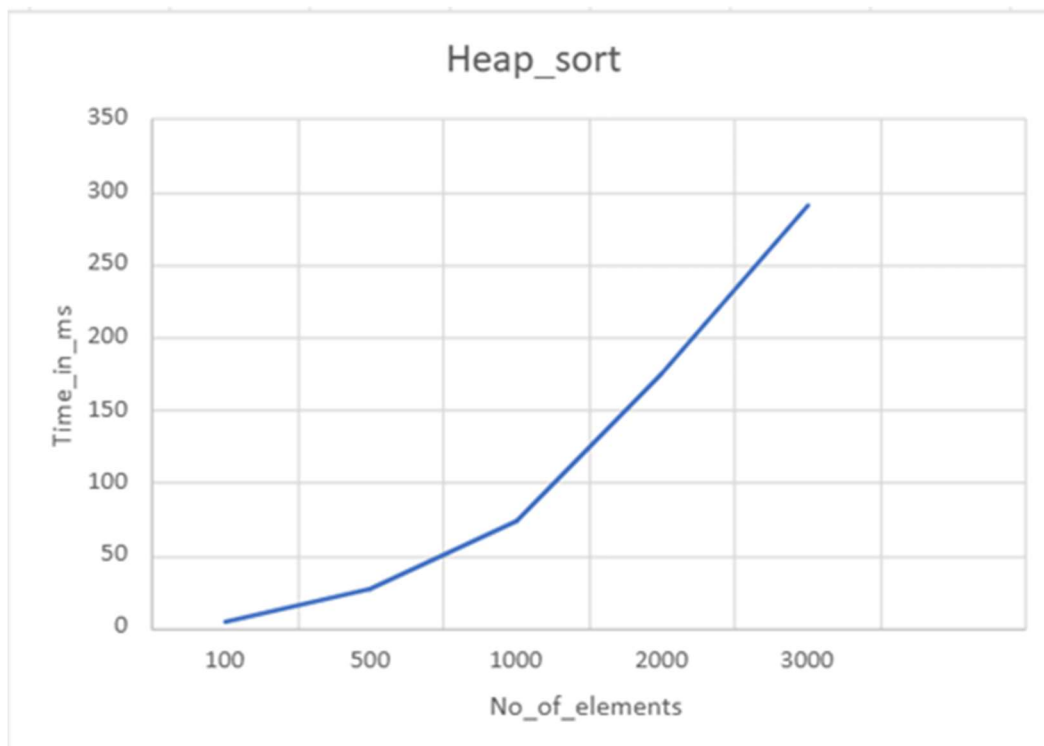
```

```
    return 0;  
}
```

OUTPUT:

```
Output  
Enter number of elements: 7  
Enter elements:  
12  
11  
13  
5  
  
7  
6  
3  
Sorted elements are:  
3 5 6 7 11 12 13  
Time taken = 3.000000 seconds
```

GRAPH:



LAB PROGRAM 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>
```

```
#define MAX 20
```

```
int knapsack(int n, int w[], int p[], int m) {  
    int table[MAX][MAX];  
    int i, j;  
  
    for(i = 0; i <= n; i++) {  
        for(j = 0; j <= m; j++) {  
  
            if(i == 0 || j == 0)  
                table[i][j] = 0;  
  
            else if(j < w[i - 1])  
                table[i][j] = table[i - 1][j];  
  
            else {  
                int include = p[i - 1] + table[i - 1][j - w[i - 1]];  
                int exclude = table[i - 1][j];  
  
                if(include > exclude)  
                    table[i][j] = include;  
                else  
                    table[i][j] = exclude;  
            }  
        }  
    }  
  
    return table[n][m];  
}
```

```
int main() {  
    int n, m, i;  
    int w[MAX], p[MAX];  
    int profit;  
  
    printf("Enter number of items: ");  
    scanf("%d", &n);  
  
    printf("Enter weights:\n");
```

```

    for(i = 0; i < n; i++) {
        scanf("%d", &w[i]);
    }

    printf("Enter profits:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &p[i]);
    }

    printf("Enter knapsack capacity: ");
    scanf("%d", &m);

    profit = knapsack(n, w, p, m);

    printf("Maximum profit = %d\n", profit);

    return 0;
}

```

OUTPUT:

Output

```

Enter number of items: 4
Enter weights:
5 3 4 1
Enter profits:
7 4 5 1
Enter knapsack capacity: 7
Maximum profit = 9

```

leetcode 801

The screenshot displays the LeetCode interface for problem 801. The left pane shows a list of submissions with the following details:

Status	Language	Runtime	Memory
Accepted	C++	6 ms	15.6 MB
Wrong Answer	C++	N/A	N/A
Wrong Answer	C++	N/A	N/A
Accepted	C++	8 ms	16.4 MB
Wrong Answer	C++	N/A	N/A

The right pane shows the C++ code for the solution:

```
1 int min(int a, int b)
2 { if (a < b)
3   return b;
4   else return a;
5 }
6
7
8 int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
9
10    int table[2][nums1Size];
11    table[0][0] = 0;
12    table[1][0] = 1;
13    int i = 1;
14    for (i = 1; i < nums1Size; i++)
15    {
16        table[0][i] = 100000;
17        table[1][i] = 100000;
18        if (nums1[i] > nums1[i-1] && nums2[i] > nums2[i-1])
19        { table[0][i] = table[0][i-1];
20          table[1][i] = 1 + table[1][i-1];
21        }
22    }
23    if (nums1[i] > nums2[i-1] && nums2[i] > nums1[i-1]) {
24        table[0][i] = min(table[0][i], table[1][i-1]);
25        table[1][i] = min(table[1][i], 1 + table[0][i-1]);
26    }
27 }
28
```

LAB PROGRAM 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
```

```
#include <stdio.h>
```

```
#define MAX 20
```

```
#define INF 999
```

```
void floyds(int a[MAX][MAX], int n) {  
    int i, j, k;  
  
    for(k = 0; k < n; k++) {  
        for(i = 0; i < n; i++) {  
            for(j = 0; j < n; j++) {  
  
                if(a[i][j] > a[i][k] + a[k][j])  
                    a[i][j] = a[i][k] + a[k][j];  
  
            }  
        }  
    }  
}
```

```
int main() {  
    int a[MAX][MAX];  
    int n, i, j;  
  
    printf("Enter number of vertices: ");  
    scanf("%d", &n);  
  
    printf("Enter adjacency matrix:\n");  
  
    for(i = 0; i < n; i++) {  
        for(j = 0; j < n; j++) {  
            scanf("%d", &a[i][j]);  
  
            if(i != j && a[i][j] == 0)  
                a[i][j] = INF;  
        }  
    }  
  
    floyds(a, n);  
}
```

```

printf("All Pair Shortest Path Matrix:\n");

for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {

        if(a[i][j] == INF)
            printf("INF ");
        else
            printf("%d ", a[i][j]);
    }

    printf("\n");
}

return 0;
}

```

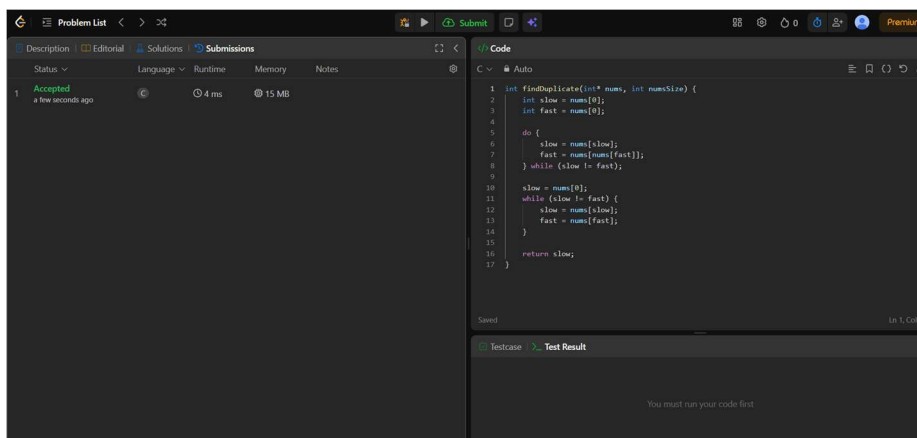
OUTPUT:

```

Output

Enter number of vertices: 4
Enter adjacency matrix:
0 5 0 10
0 0 3 0
0 0 0 1
0 0 0 0
All Pair Shortest Path Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

```



LAB PROGRAM 8(A):

Find Minimum cost spanning tree of a given undirected graph using Prims Algorithm

```
#include <stdio.h>
```

```
#define MAX 20
```

```
#define INF 999
```

```
int prims(int n, int cost[MAX][MAX]) {
    int visited[MAX] = {0};
    int mincost = 0;
    int i, j, u, v, min;

    visited[1] = 1;
    int edges = 1;

    while(edges < n) {
        min = INF;
        u = -1;
        v = -1;

        for(i = 1; i <= n; i++) {
            if(visited[i]) {
                for(j = 1; j <= n; j++) {
                    if(!visited[j] && cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }

        if(u != -1 && v != -1) {
            visited[v] = 1;
            mincost += min;

            printf("%d - %d : %d\n", u, v, min);
            edges++;
        }
        else {
            break;
        }
    }
}
```

```

    return mincost;
}

int main() {
    int n, cost[MAX][MAX];
    int i, j, result;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost adjacency matrix:\n");
    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    result = prims(n, cost);

    printf("Minimum Cost = %d\n", result);

    return 0;
}

```

OUTPUT:

Output

```

Enter number of vertices: 4
Enter cost adjacency matrix:
0 4 5 0
4 0 1 4
5 1 0 5
0 4 5 0
1 - 2 : 4
2 - 3 : 1
2 - 4 : 4
Minimum Cost = 9

```

LAB PROGRAM 8(b) :

Find Minimum cost spanning tree of a given undirected graph using Krushkal's Algorithm

```
#include <stdio.h>
```

```
#define MAX 20
```

```
#define INF 999
```

```
int set[MAX];
```

```
int find(int x) {  
    return set[x];  
}
```

```
void union_set(int u, int v, int n) {  
    int i;  
    int old = set[v];  
  
    for(i = 1; i <= n; i++) {  
        if(set[i] == old)  
            set[i] = set[u];  
    }  
}
```

```
int kruskal(int n, int cost[MAX][MAX]) {  
    int i, j, k = 0;  
    int min, u = 0, v = 0;  
    int mincost = 0;  
  
    for(i = 1; i <= n; i++)  
        set[i] = i;  
  
    while(k < n - 1) {  
        min = INF;  
  
        for(i = 1; i <= n; i++) {  
            for(j = 1; j <= n; j++) {  
                if(cost[i][j] < min) {  
                    min = cost[i][j];  
                    u = i;  
                    v = j;  
                }  
            }  
        }  
    }  
}
```

```

    int root_u = find(u);
    int root_v = find(v);

    if(root_u != root_v) {
        printf("%d - %d : %d\n", u, v, min);
        mincost += min;
        union_set(u, v, n);
        k++;
    }

    cost[u][v] = cost[v][u] = INF;
}

return mincost;
}

int main() {
    int n, cost[MAX][MAX];
    int i, j, result;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost adjacency matrix:\n");

    for(i = 1; i <= n; i++) {
        for(j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);

            if(cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    result = kruskal(n, cost);

    printf("Minimum Cost = %d\n", result);

    return 0;
}

```

OUTPUT:

Output

```
Enter number of vertices: 4
Enter cost adjacency matrix:
0 2 3 0
2 0 1 4
3 1 0 5
0 4 5 0
2 - 3 : 1
1 - 2 : 2
2 - 4 : 4
Minimum Cost = 7
```

LAB PROGRAM 9:

Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>

#define MAX 20

void knap(int n, double w[], double p[], double cap) {
    double ratio[MAX];
    double temp;
    double profit = 0.0, rem = cap;
    int i, j;

    for(i = 0; i < n; i++)
        ratio[i] = p[i] / w[i];

    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(ratio[j] < ratio[j + 1]) {

                temp = ratio[j];
                ratio[j] = ratio[j + 1];
                ratio[j + 1] = temp;

                temp = w[j];
                w[j] = w[j + 1];
                w[j + 1] = temp;

                temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }

    for(i = 0; i < n; i++) {
        if(w[i] <= rem) {
            rem -= w[i];
            profit += p[i];
        } else {
            profit += p[i] * (rem / w[i]);
            break;
        }
    }
}
```

```

    printf("%.2f", profit);
}

int main() {
    int n, i;
    double w[MAX], p[MAX], cap;

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter weights:\n");
    for(i = 0; i < n; i++)
        scanf("%lf", &w[i]);

    printf("Enter profits:\n");
    for(i = 0; i < n; i++)
        scanf("%lf", &p[i]);

    printf("Enter capacity: ");
    scanf("%lf", &cap);

    knap(n, w, p, cap);

    return 0;
}

```

OUTPUT:

Output
Enter number of items: 4
Enter weights:
2 4 6 8
Enter profits:
1 2 3 4
Enter capacity: 10
5.00

LAB PROGRAM 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
```

```
#define V 4
```

```
#define INF 999
```

```
int mini(int d[], int visited[]) {  
    int min = INF, index = -1;  
    int i;
```

```
    for(i = 0; i < V; i++) {  
        if(visited[i] == 0 && d[i] <= min) {  
            min = d[i];  
            index = i;  
        }  
    }
```

```
    return index;  
}
```

```
void dijkstra(int g[V][V], int src) {  
    int d[V];  
    int visited[V];  
    int i, j, u;
```

```
    for(i = 0; i < V; i++) {  
        d[i] = INF;  
        visited[i] = 0;  
    }
```

```
    d[src] = 0;
```

```
    for(i = 0; i < V - 1; i++) {
```

```
        u = mini(d, visited);  
        visited[u] = 1;
```

```
        for(j = 0; j < V; j++) {  
            if(!visited[j] && g[u][j] &&  
                d[u] != INF &&
```

```

        d[u] + g[u][j] < d[j]) {
            d[j] = d[u] + g[u][j];
        }
    }
}

printf("Vertex \t Distance from Source\n");
for(i = 0; i < V; i++) {
    printf("%d \t %d\n", i, d[i]);
}
}

int main() {

    int g[V][V] = {
        {0, 1, 4, 0},
        {1, 0, 2, 6},
        {4, 2, 0, 3},
        {0, 6, 3, 0}
    };

    int src;

    printf("Enter source vertex (0-3): ");
    scanf("%d", &src);

    dijkstra(g, src);

    return 0;
}

```

OUTPUT:

Output

```

Enter source vertex (0-3): 0
Vertex    Distance from Source
0         0
1         1
2         3
3         6

```

LAB PROGRAM 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
int board[20];
int count = 0;

bool isSafe(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col) {
            return false;
        }
        if (abs(board[i] - col) == abs(i - row)) {
            return false;
        }
    }
    return true;
}

void solveNQueens(int row, int n) {
    if (row == n) {
        count++;
        printf("\nSolution %d:\n", count);
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i] == j) printf("Q ");
                else printf(". ");
            }
            printf("\n");
        }
        return;
    }

    for (int col = 0; col < n; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            solveNQueens(row + 1, n);
        }
    }
}
```

```

int main() {
    int n;
    printf("Enter the size of the board (N): ");
    scanf("%d", &n);

    solveNQueens(0, n);

    printf("\nTotal Solutions Found: %d\n", count);
    return 0;
}

```

OUTPUT:

Output

Enter the size of the board (N): 4

Solution 1:

```

. Q . .
. . . Q
Q . . .
. . Q . |

```

Solution 2:

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Total Solutions Found: 2